

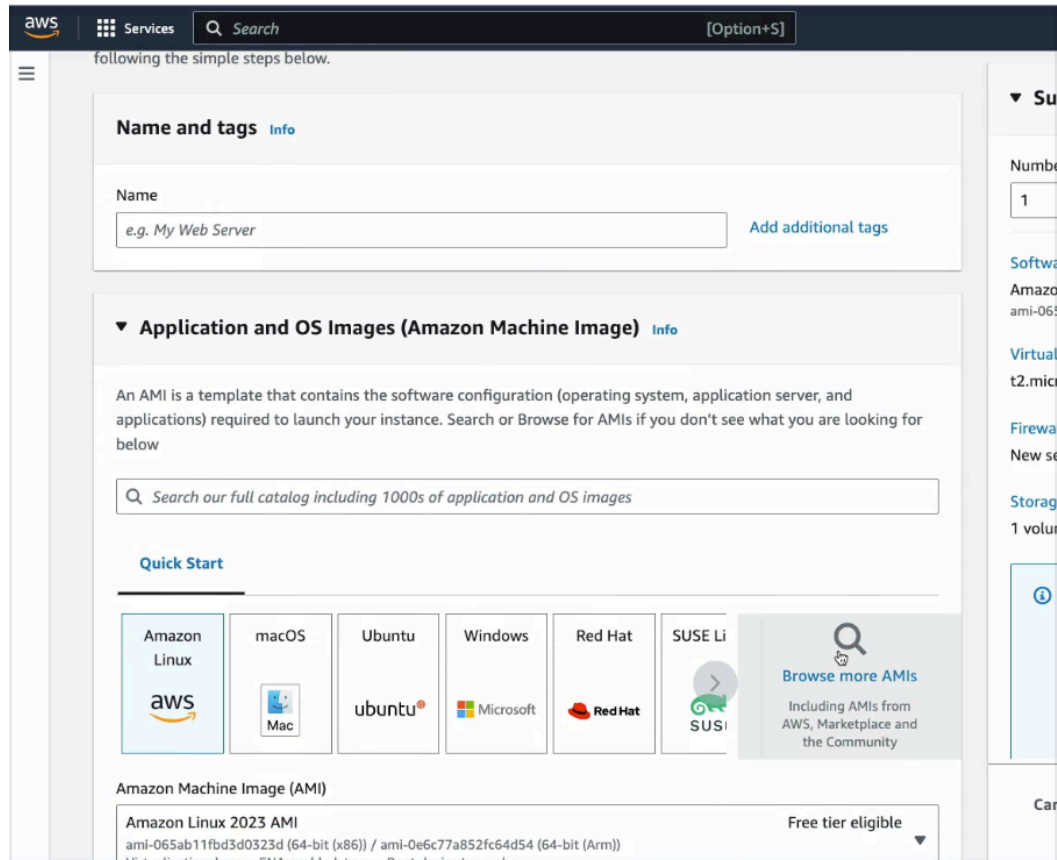
Module 12 - chapter 13

Automate Provisioning EC2 with Terraform - Part 2

4. Then we will create the EC2 instance

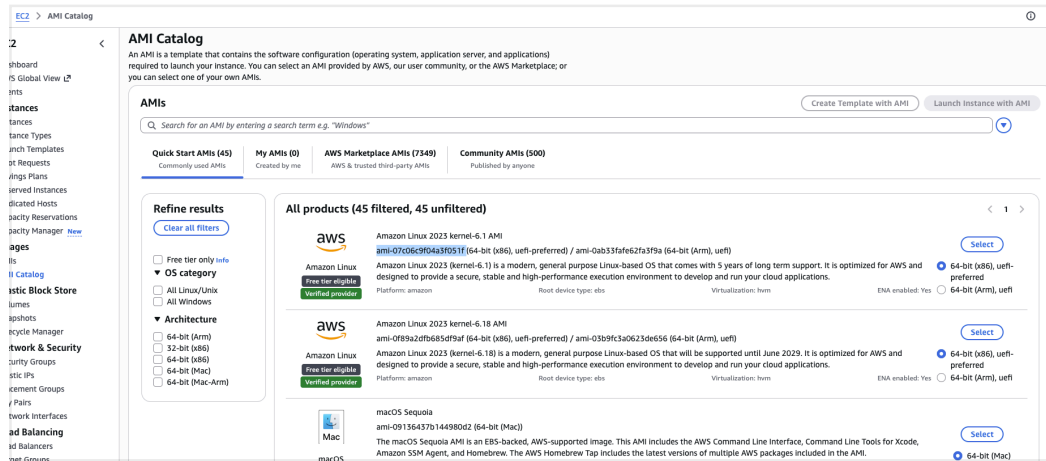
Amazon Machine Image - AMI for EC2

The first thing when creating an EC2 instance that we need to have using the console is an Amazon Machine Image - AMI



Which is a template (a full VM) that contains the software configuration that we will use to be able to launch the server: OS, application server, and applications.

We will use Amazon linux AMI, so I need its ID to reference it in TF -> ami-07c06c9f04a3f051f

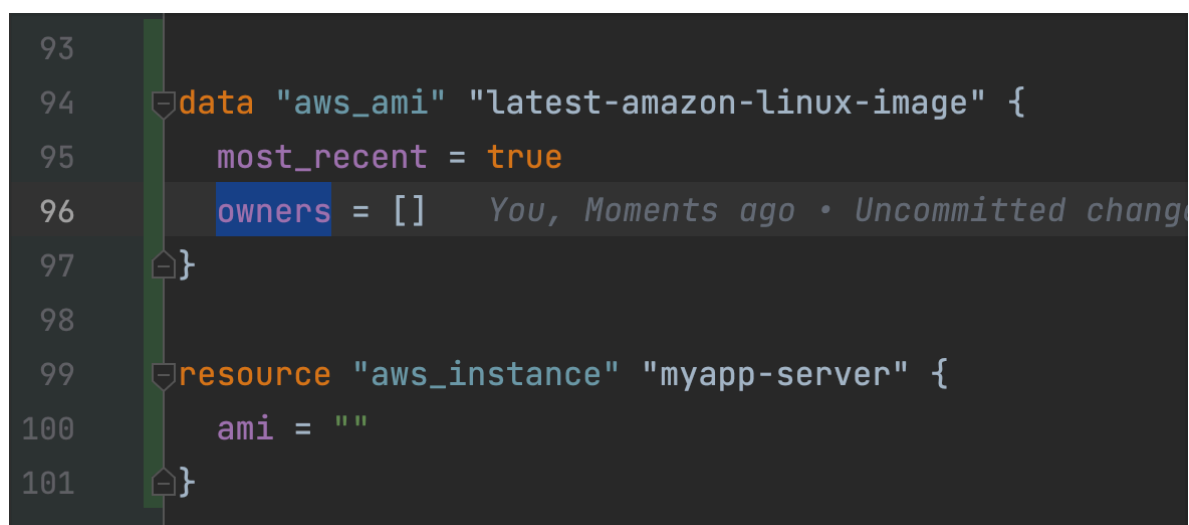


The same image can have a different ID if we are using a different REGION.

Also the Images IDs can change is is dynamic -> is better not to hardcode the ID in TF, so it is better to get the ID from AWS

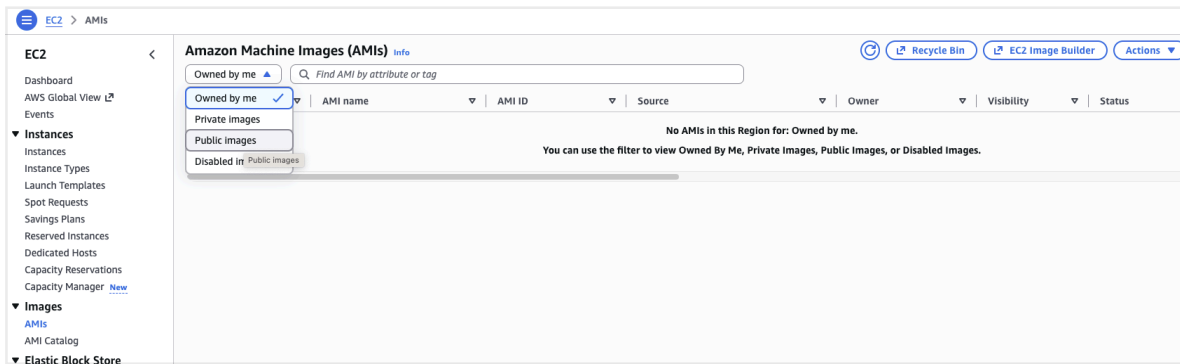


To get the AMI ID dynamically we use a data query

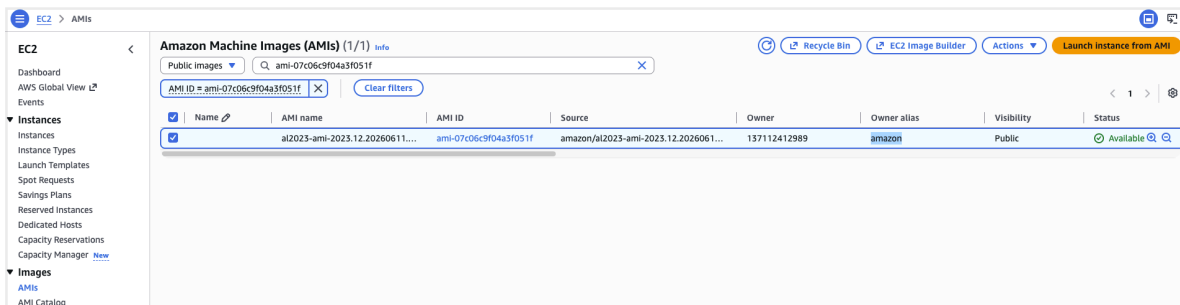


We configure the data query to look for the most recent AMI. And we also pass the attribute of 'owner' which is the company who created the image

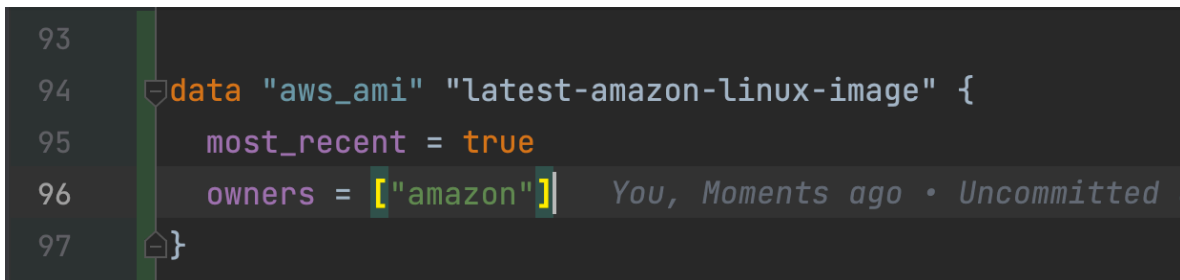
-> go to AMIs and filter by 'public images'



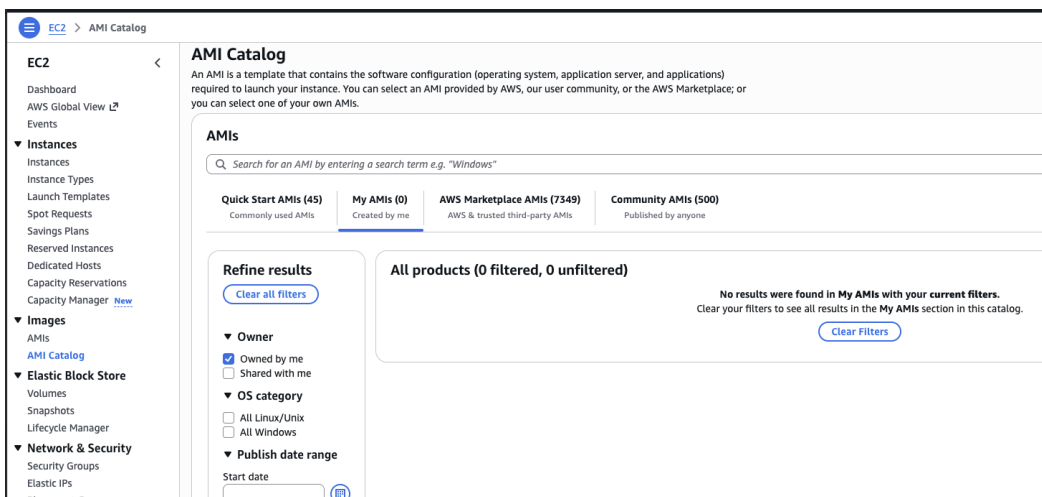
And paste the current id of the AMI we want to use in the 'search' box



And we can see that the owner is 'amazon'



If I have an EC2 instance and then I make changes to it then I can make my own image out of it and they will be displayed in the tab 'My AMIs'



So far our data query says: AWS give me the "latest" AMI created by "amazon"

but I still need to be more specific and for that we use "filter" attribute

```
93
94 data "aws_ami" "latest-amazon-linux-image" {
95     most_recent = true
96     owners      = ["amazon"]
97     filter {
98         name = "name"
99         values = []
100     }
101 }
```

So we will filter by the "name" of the AMI (see line 98)

The screenshot shows the AWS Marketplace console with the 'Quick Start AMIs (45)' tab selected. The 'Refine results' sidebar on the left shows filters for 'OS category' (All Linux/Unix, All Windows) and 'Architecture' (64-bit (Arm), 32-bit (x86), 64-bit (x86), 64-bit (Mac), 64-bit (Mac-Arm)). The main area displays 'All products (45 filtered, 45 unfiltered)'. Two AMIs are visible: 'Amazon Linux 2023 kernel-6.1 AMI' and 'Amazon Linux 2023 kernel-6.18 AMI'. Both are marked as 'Free tier eligible' and 'Verified provider'.

As we can see the names of some AMIs can be very similar, so we will take part of the name.

Problem -> when I check the images with similar name, I get a long list:

```
aws ec2 describe-images \
  --owners amazon \
  --filters "Name=name,Values=al2023-ami-*kernel-6.1*" \
  --query "Images[*].Name" \
  --output table
```

```
aws ec2 describe-images \
  --owners amazon \
  --region eu-west-2 \
  --filters \
    "Name=name,Values=al2023-ami-*kernel-6.1*" \
    "Name=architecture,Values=x86_64" \
```

```
"Name=root-device-type,Values=ebs" \
"Name=virtualization-type,Values=hvm" \
--query "Images[*].Name" \
--output table
```

Long list of AMIs that match the filter:

DescribeImages
al2023-ami-ecs-hvm-2023.0.20240702-kernel-6.1-x86_64
al2023-ami-ecs-gpu-hvm-2023.0.20250321-kernel-6.1-x86_64-ebs
al2023-ami-ecs-hvm-2023.0.20260120-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20260514-kernel-6.1-x86_64
al2023-ami-ecs-neuron-hvm-2023.0.20250730-kernel-6.1-x86_64
al2023-ami-minimal-2023.11.20260406.2-kernel-6.12-x86_64
al2023-ami-ecs-neuron-hvm-2023.0.20250422-kernel-6.1-x86_64
al2023-ami-minimal-2023.11.20260427.1-kernel-6.18-x86_64
al2023-ami-ecs-gpu-hvm-2023.0.20260506-kernel-6.1-x86_64-ebs
al2023-ami-2023.11.20260526.0-kernel-6.12-x86_64
al2023-ami-ecs-neuron-hvm-2023.0.20260129-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20250214-kernel-6.1-x86_64
al2023-ami-ecs-neuron-hvm-2023.0.20260527-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20241010-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20240625-kernel-6.1-x86_64
al2023-ami-ecs-neuron-hvm-2023.0.20241217-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20251111-kernel-6.1-x86_64
al2023-ami-minimal-2023.11.20260511.1-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20241017-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20241217-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20260108-kernel-6.1-x86_64
al2023-ami-minimal-2023.12.20260608.0-kernel-6.12-x86_64
al2023-ami-ecs-neuron-hvm-2023.0.20250403-kernel-6.1-x86_64
al2023-ami-ecs-neuron-hvm-2023.0.20240712-kernel-6.1-x86_64
al2023-ami-minimal-2023.11.20260505.0-kernel-6.1-x86_64
al2023-ami-ecs-gpu-hvm-2023.0.20250716-kernel-6.1-x86_64-ebs
al2023-ami-ecs-gpu-hvm-2023.0.20260223-kernel-6.1-x86_64-ebs
al2023-ami-ecs-hvm-2023.0.20250129-kernel-6.1-x86_64
al2023-ami-ecs-neuron-hvm-2023.0.20260114-kernel-6.1-x86_64
al2023-ami-ecs-neuron-hvm-2023.0.20241010-kernel-6.1-x86_64
al2023-ami-ecs-gpu-hvm-2023.0.20250828-kernel-6.1-x86_64-ebs
al2023-ami-ecs-neuron-hvm-2023.0.20241031-kernel-6.1-x86_64
al2023-ami-2023.11.20260526.0-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20250327-kernel-6.1-x86_64
al2023-ami-ecs-gpu-hvm-2023.0.20250923-kernel-6.1-x86_64-ebs
al2023-ami-ecs-neuron-hvm-2023.0.20251209-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20250610-kernel-6.1-x86_64
al2023-ami-2023.11.20260509.0-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20250121-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20250828-kernel-6.1-x86_64
al2023-ami-ecs-gpu-hvm-2023.0.20260514-kernel-6.1-x86_64-ebs
al2023-ami-ecs-gpu-hvm-2023.0.20250618-kernel-6.1-x86_64-ebs
al2023-ami-2023.11.20260526.0-kernel-6.18-x86_64
al2023-ami-ecs-neuron-hvm-2023.0.20251015-kernel-6.1-x86_64
al2023-ami-ecs-hvm-2023.0.20251209-kernel-6.1-x86_64
al2023-ami-ecs-gpu-hvm-2023.0.20250714-kernel-6.1-x86_64-ebs
al2023-ami-minimal-2023.12.20260611.0-kernel-6.18-x86_64

Solution is to check VIA SSM with this code:

```
-> aws ssm get-parameters \
--names /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-
x86_64 \
--query 'Parameters[0].Value' --output text
```

And it works! I only get one id back  and it matches the one from the console

```
ami-07c06c9f04a3f051f  
(END)
```

— SMM tangent —

better approach than filtering `describe-images`. It uses AWS Systems Manager Parameter Store which AWS keeps up to date with the latest AMI IDs.

Run it and it will return a single AMI ID like `ami-xxxxxxxxxxxxxxxx` — that's the latest Amazon Linux 2023 kernel default for `x86_64`.

Then in Terraform you can use it as a data source instead of a filter:

```
data "aws_ssm_parameter" "latest-amazon-linux-image" {  
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-  
x86_64"  
}  
  
resource "aws_instance" "myapp-server" {  
  ami = data.aws_ssm_parameter.latest-amazon-linux-image.value  
  ...  
}
```

Advantages over `aws_ami` filter:

- Always gets the latest AMI automatically
- No need to maintain filter patterns
- Simpler and more reliable
- AWS maintains these SSM parameters officially

This is actually the recommended way to reference Amazon Linux AMIs in Terraform.

Q:

what is SSM

A:

- SSM = AWS Systems Manager

It's a broad AWS service that helps you manage your infrastructure. It has many features but the one relevant here is Parameter Store — a secure storage for configuration data and secrets.

AWS uses Parameter Store to publish and maintain official paths to their latest

AMI IDs, like:

/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64

AWS keeps these paths updated automatically whenever they release a new AMI, so when you query it you always get the latest one.

Other things SSM is used for:

- Storing secrets and config values (database passwords, API keys)
- Running commands on EC2 instances remotely without SSH
- Patching EC2 instances automatically
- Session Manager — connect to EC2 instances without opening port 22

But for your current use case, just think of it as "***a reliable place to look up the latest AMI ID.***"

— end tangent —

Nana used two filters in TF "name" and "virtualisation-type". We can use as many filters as we want

```
76 data "aws_ami" "latest-amazon-linux-image" {
77     most_recent = true
78     owners = ["amazon"]
79     filter {
80         name = "name"
81         values = ["amzn2-ami-kernel-*-x86_64-gp2"]
82     }
83     filter {
84         name = "virtualization-type"
85         values = ["hvm"]
86     }
87 }
```

My AWS is displaying a different name as the one from Nana's video

AMIs

Search for an AMI by entering a search term e.g. "Windows"

Quick Start AMIs (45) | My AMIs (0) | AWS Marketplace AMIs (7349) | Community AMIs (500)

Commonly used AMIs | Created by me | AWS & trusted third-party AMIs | Published by anyone

Refine results

Clear all filters

Free tier only [info](#)

OS category

All Linux/Unix

All Windows

Architecture

64-bit (Arm)

32-bit (x86)

All products (45 filtered, 45 unfiltered)

aws

Amazon Linux

Free tier eligible

Verified provider

Amazon Linux 2023 kernel-6.1 AMI

ami-07c06c9f04a3f051f (64-bit (x86), uefi-preferred) / ami-0ab33fafe62fa3f9a (64-bit (Arm), uefi-preferred)

Amazon Linux 2023 (kernel-6.1) is a modern, general purpose Linux-based OS that comes with 5 years of support designed to provide a secure, stable and high-performance execution environment to develop and deploy your applications.

Platform: amazon | Root device type: ebs | Virtualization type: hvm

aws

Amazon Linux

Free tier eligible

Verified provider

Amazon Linux 2023 kernel-6.18 AMI

ami-0f89a2dfb685df9af (64-bit (x86), uefi-preferred) / ami-03b9fc3a0623de656 (64-bit (Arm), uefi-preferred)

Amazon Linux 2023 (kernel-6.18) is a modern, general purpose Linux-based OS that will be supported for 5 years.

And that name is throwing an error when I use it in TF
-> terraform plan

```

terraform main.tf
94 data "aws_ami" "latest-amazon-linux-image" {
95   most_recent = true
96   owners = ["amazon"]
97   filter {
98     name = "name"
99     values = ["Amazon Linux 2023 kernel-6.1 AMI"]
100   }
101   filter {
102     name = "virtualization-type"
103     values = ["hvm"]
104   }
105 }

data aws_ami latest-amazon-linux-image filter values

Terminal: zsh
+ tags_all = {
+   "Name" = "dev-vpc"
+ }

Plan: 5 to add, 0 to change, 0 to destroy.

Error: Your query returned no results. Please change your search criteria and try again.

with data.aws_ami.latest-amazon-linux-image,
on main.tf line 94, in data "aws_ami" "latest-amazon-linux-image":
94: data "aws_ami" "latest-amazon-linux-image" {

```

So I check the actual name and it is: al2023-ami-2023.12.20260611.0-kernel-6.1-x86_64

Amazon Machine Images (AMIs) (1/2) [info](#)

Public images | Search

AMI ID = ami-0d0493a60f47102f0 | AMI ID = ami-07c06c9f04a3f051f | Clear filters

	Name	AMI name	AMI ID	Source	Owner	Owner alias
☐		al2023-ami-ecs-neuron-hvm-2023.0.20260615-kernel-6.1-...	ami-0d0493a60f47102f0	amazon/al2023-ami-ecs-neuron-hvm-2...	591542846629	amazon
☒		al2023-ami-2023.12.20260611.0-kernel-6.1-x86_64	ami-07c06c9f04a3f051f	amazon/al2023-ami-2023.12.2026061...	137112412989	amazon

When I use the full actual name in the filter then the 'output' prints the ID that is expected:

```

Changes to Outputs:
+ aws_ami_id = {
+   allow_unsafe_filter = null
+   architecture       = "x86_64"
+   arn                 = "arn:aws:ec2:eu-west-2::image/ami-07c06c9f04a3f051f"
+   block_device_mappings = [
+     + {
+       + device_name = "/dev/xvda"
+       + ebs         = {
+         + delete_on_termination = "true"
+         + encrypted             = "false"

```

If Instead of the full name I use regex to have a more generic name for example "al2023-ami-*-kernel-*-x86_64" then the output shows us that the AMI selected is a 'neuron' and this is not the AMI I want:

```

98     name = "name"
99     values = ["al2023-ami-*-kernel-*-x86_64"] You, 4 minutes ago • Uncommitted changes
100   }
101   filter {
    data aws_ami latest-amazon-linux-image > filter > values
Terminal: zsh x + v
Plan: 5 to add, 0 to change, 0 to destroy.

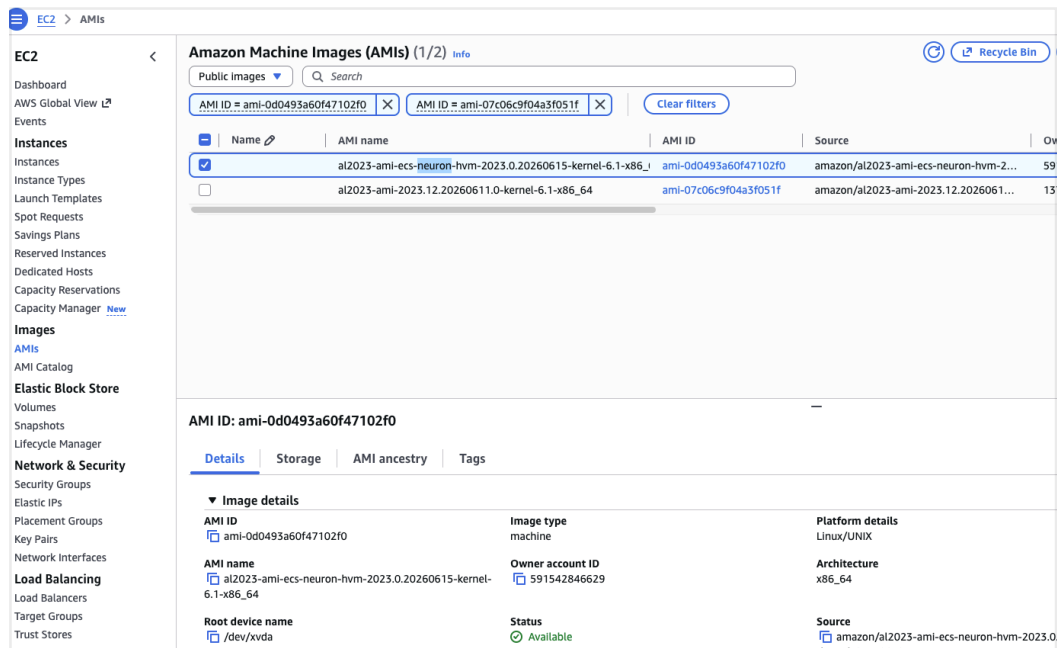
Changes to Outputs:
+ aws_ami_id = {
+   allow_unsafe_filter = null
+   architecture       = "x86_64"
+   arn                 = "arn:aws:ec2:eu-west-2::image/ami-0d0493a60f47102f0"
+   block_device_mappings = [
+     + {
+       + device_name = "/dev/xvda"

```

```

98     name = "name"
99     values = ["al2023-ami-*-kernel-*-x86_64"] You, 4 minutes ago • Uncommitted changes
100   }
101   filter {
    data aws_ami latest-amazon-linux-image > filter > values
Terminal: zsh x + v
+ values = [
+   + "hvm",
+ ]
+ },
+ ]
+ hypervisor      = "xen"
+ id              = "ami-0d0493a60f47102f0"
+ image_id        = "ami-0d0493a60f47102f0"
+ image_location  = "amazon/al2023-ami-ecs-neuron-hvm-2023.0.20260615-kernel-6.1-x86_64"
+ image_owner_alias = "amazon"
+ image_type      = "machine"
+ imds_support    = "v2.0"
+ include_deprecated = false

```



— tangent —

Q:

al2023-ami-ecs-neuron-hvm-2023.0.20260615-kernel-6.1-x86_64 VS al2023-ami-2023.12.20260611.0-kernel-6.1-x86_64

A:

● The second one is what you want: al2023-ami-2023.12.20260611.0-kernel-6.1-x86_64

Here's why:

al2023-ami-ecs-neuron-hvm-...

- ecs — specialized for Elastic Container Service
- neuron — specialized for AWS Inferentia/Trainium chips (AWS's AI/ML hardware)
- Not a general purpose AMI

al2023-ami-2023.12.20260611.0-kernel-6.1-x86_64

- Standard Amazon Linux 2023
- **General purpose**
- kernel 6.1
- x86_64 architecture
- This matches exactly what you described earlier

This confirms the SSM parameter is pointing to the right AMI. You can now use it in your Terraform data source.

— end tangent —

To get the AMI I want (not 'neutron') I can use this regex -> "al2023-ami-2023*-kernel-*-x86_64"

```
93
94 data "aws_ami" "latest-amazon-linux-image" {
95     most_recent = true
96     owners = ["amazon"]
97     filter {
98         name = "name"
99         # values = ["al2023-ami-*-kernel-*-x86_64"] // it returns a neutron image, so it needs more filtering
100        values = ["al2023-ami-2023*-kernel-*-x86_64"] // it returns the image we want You, Moments ago • Uncommitted changes
101    }
102    filter {
103        name = "virtualization-type"
104        values = ["hvm"]
105    }
106 }
107
108 output "latest-amazon-linux-image-id" {
109     value = data.aws_ami.latest-amazon-linux-image.id
110 }
```

But I will use "aws_ssm_parameter" instead of "aws_ami" see lines 107 - 109 because "This is actually the recommended way to reference Amazon Linux AMIs in Terraform" according to AI in the notes above.

```
-> data "aws_ssm_parameter" "latest-amazon-linux-image" {
    name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}
```

```
93
94 #data "aws_ami" "latest-amazon-linux-image" {
95     # most_recent = true
96     # owners = ["amazon"]
97     # filter {
98     #     name = "name"
99     #     values = ["al2023-ami-kernel-*-x86_64"]
100     # }
101     # filter {
102     #     name = "virtualization-type"
103     #     values = ["hvm"]
104     # }
105 #}
106
107 data "aws_ssm_parameter" "latest-amazon-linux-image" {
108     name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64" You, 34 minutes
109 }
110
111 resource "aws_instance" "myapp-server" {
112     ami = ""
113 }
```

Now we can use the value returned by the data query in the creation of the EC2 resource:

```

107 data "aws_ssm_parameter" "latest-amazon-linux-image" {
108     name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
109 }
110
111 resource "aws_instance" "myapp-server" {
112     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
113 }

```

— tangent —

Q:

and with this

```

-> data "aws_ssm_parameter" "latest-amazon-linux-image" {
    name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-
x86_64"
}

```

I will get the latest AMI any time I run TF even when the image has been updated

A:

- Yes, exactly! Every time you run terraform apply it will query the SSM parameter at that moment and get whatever the latest AMI ID is at that point in time.

However there is one thing to be aware of — if AWS updates the AMI and the SSM parameter points to a new ID, Terraform will detect the change and replace your EC2 instance (destroy old, create new).

This can be unexpected in production.

For learning/dev environments — this is fine, always get the latest.

For **production** — many teams pin to a specific AMI ID to avoid unexpected instance replacements, and update deliberately when they're ready.

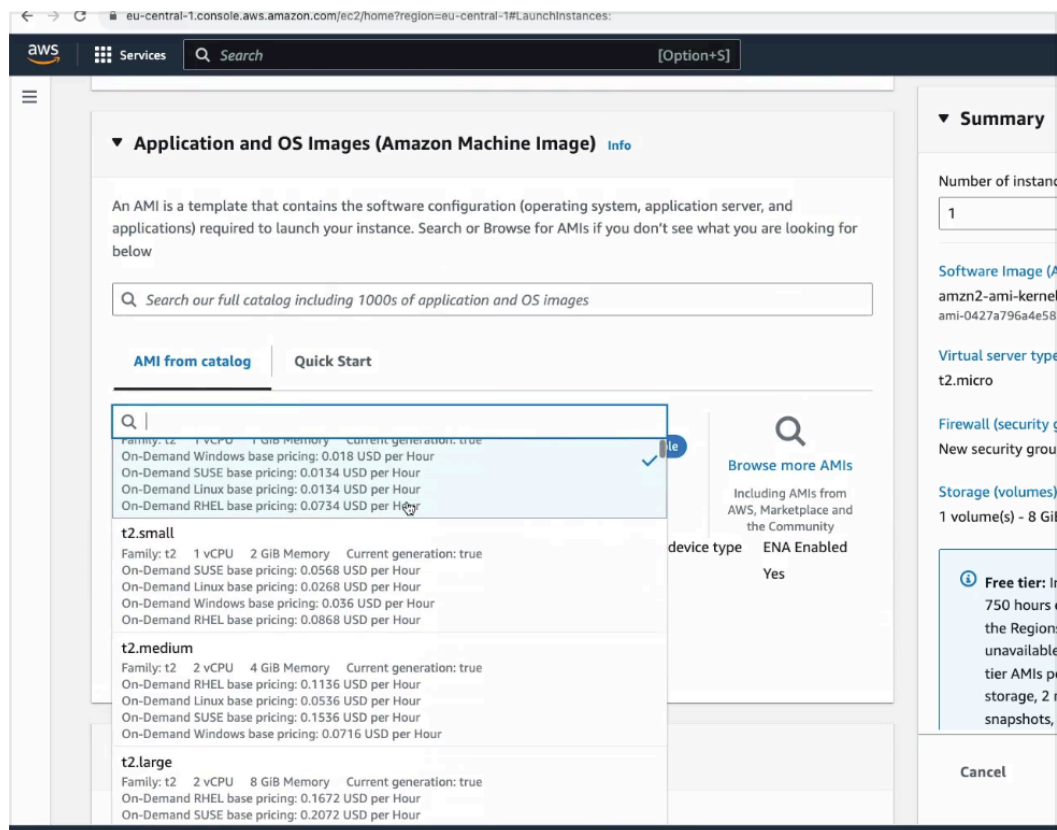
But for what you're doing now, the SSM approach is perfect.

— end tangent —

Create EC2 Instance

4. Then we will create the EC2 instance

We need to add the instance type e.g. t2.small, etc



```

117 resource "aws_instance" "myapp-server" {
118     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
119     instance_type = "t3.micro"
120 }

```

And we can make the instance type configurable incase we want one type in dev but another type in UAT.

```
terraform.tfvars
main.tf
9 variable my_ip {}
10 variable instance_type {}
11
12 resource "aws_vpc" "myapp-vpc" {...}
13
14 resource "aws_subnet" "myapp-subnet-1" {...}
15
16 #...
17
18 resource "aws_internet_gateway" "myapp-igw" {...}
19
20 #...
21
22 resource "aws_default_route_table" "main-rtb" {...}
23
24 resource "aws_default_security_group" "default-sg" {...}
25
26 #...
27
28 data "aws_ssm_parameter" "latest-amazon-linux-image" {...}
29
30 resource "aws_instance" "myapp-server" {
31   ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
32   instance_type = var.instance_type
33 }
34
35 vpc_cidr_block = "10.0.0.0/16"
36 subnet_cidr_block = "10.0.10.0/24"
37 avail_zone = "eu-west-2a"
38 env_prefix = "dev"
39 my_ip = "140.228.47.252/32"
40 instance_type = "t2.micro"
```

Optional arguments

```
main.tf
76
77 data "aws_ami" "latest-amazon-linux-image" {
78   most_recent = true
79   owners = ["amazon"]
80   filter {
81     name = "name"
82     values = ["amzn2-ami-kernel-..."]
83   }
84   filter {
85     name = "virtualization-type"
86     values = ["hvm"]
87   }
88 }
89
90 output "aws_ami_id" {
91   value = data.aws_ami.latest-amazon-linux-image.id
92 }
93
94 resource "aws_instance" "myapp-server" {
95   ami = data.aws_ami.latest-amazon-linux-image.id
96   instance_type = var.instance_type
97 }
```

Optional arguments

- other arguments are optional
- default components are assigned, if we don't specify them

If we don't specify the subnetId and securityGroupId then the EC2 will be launch in our AWS account default VPC.

We want the EC2 to be launched in the VPC we created in Tf and in a specific subnet with a specific SG.

```

19 resource "aws_subnet" "myapp-subnet-1" {...}
27
28 #...
39
40 resource "aws_internet_gateway" "myapp-igw" {...}
46
47 #...
51
52 resource "aws_default_route_table" "main-rtb" {...}
63
64 resource "aws_default_security_group" "default-sg" {...}
94
95 #...
114 data "aws_ssm_parameter" "latest-amazon-linux-image" {...}
117
118 resource "aws_instance" "myapp-server" {
119     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
120     instance_type = var.instance_type
121
122     subnet_id = aws_subnet.myapp-subnet-1.id
123     vpc_security_group_ids = [aws_default_security_group.default-sg.id]
124 }

```

We can assign multiple sg to an EC2 so the attribute "vpc_security_group_ids" is a list.

And we also add the AZ

```

118 resource "aws_instance" "myapp-server" {
119     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
120     instance_type = var.instance_type
121
122     subnet_id = aws_subnet.myapp-subnet-1.id
123     vpc_security_group_ids = [aws_default_security_group.default-sg.id]
124     availability_zone = var.avail_zone
125 }

```

We now add the attribute "associate_public_ip_address = true" to assign a public IP address to our EC2 instance when it launches.

Without it (false):

- Our EC2 instance only gets a private IP (e.g. 10.0.1.5)
- Only reachable from within the VPC
- Cannot be accessed from the internet

With it (true):

- Our EC2 instance gets both a private IP and a public IP (e.g. 54.123.45.67)

- Reachable from the internet (assuming your security group and route table allow it)

Important things to know:

- The public IP is not static — if we stop and start the instance, it gets a new public IP
- If we need a permanent public IP, use an Elastic IP instead
- This only works if our subnet is in a VPC that has an Internet Gateway attached and a route to it — which we already have set up

So in our case, adding this to our EC2 resource makes it reachable from the internet, which combined with our security group rules controls who can actually connect to it.

```
118 resource "aws_instance" "myapp-server" {
119     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
120     instance_type = var.instance_type
121
122     subnet_id = aws_subnet.myapp-subnet-1.id
123     vpc_security_group_ids = [aws_default_security_group.default-sg.id]
124     availability_zone = var.avail_zone
125
126     associate_public_ip_address = true
127
128 }
```

So now the browser and my laptop (via ssh) can reach the server using its public IP address

Also, to SSH into the server we use Keys. In the past during the bootcamp we created Key pair for EC2 instance and AWS gave us a PEM file with the keys info.

EC2

Resources

You are using the following Amazon EC2 resources in the Europe (London) Region:

Instances (running)	0	Auto Scaling Groups
Dedicated Hosts	0	Elastic IPs
Key pairs	2	Load balancers
Security groups	3	Snapshots

Launch instance

To get started, launch an Amazon EC2 instance, which is a virtual server in the cloud.

[Launch instance](#) [Migrate a server](#)

Note: Your instances will launch in the Europe (London) Region

I currently have two

Name	Type	Created	Fingerprint	ID
docker-server	rsa	2026/03/24 14:43 GMT+0	ce67:11:40:9f:bb:51:a8:71:ae:60:2e:6e:ad:1c:63:e5:...	key-082c43a1d6cc13b7e
MyKpCli	rsa	2026/04/27 15:42 GMT+1	f0:9b:12:93:ec:90:dc:c7:ae:63:79:4b:22:6b:40:aca9:...	key-0955aa307df1b8f14

So in the UI here we create a new key pair

Create key pair

Key pair

A key pair, consisting of a private key and a public key, is a set of security credentials that you use to prove your identity when connecting to an instance.

Name

Enter key pair name

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ RSA ☐ ED25519

Private key file format

☒ .pem
For use with OpenSSH

☐ .ppk
For use with PuTTY

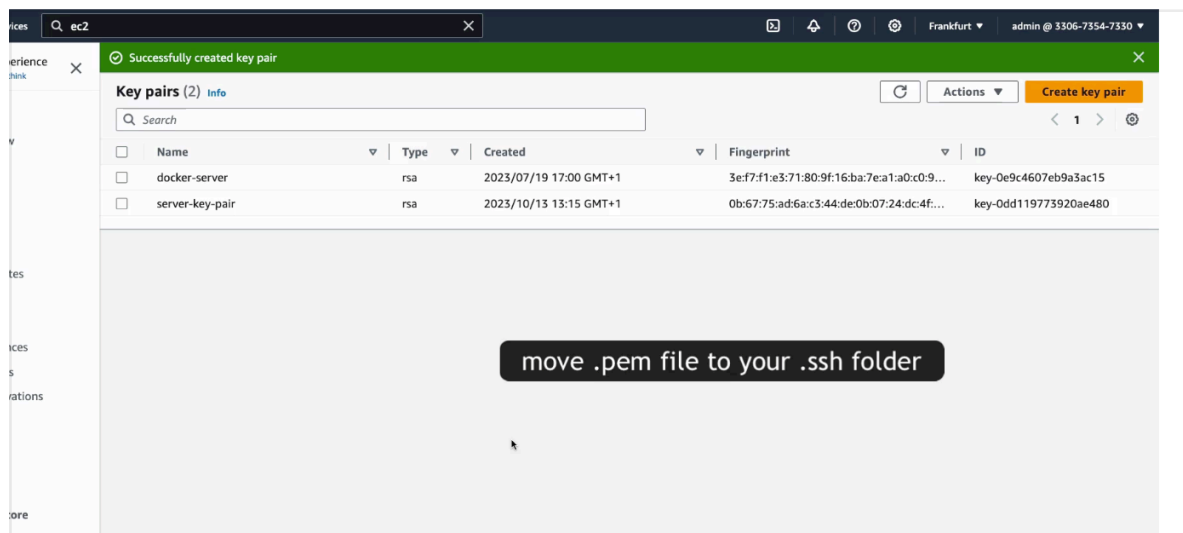
Tags - optional

No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tags.

If we create the key pair using AWS then the PEM file is downloaded to downloads folder so we need to move it to our laptops .ssh folder as it is safer



AI says is better to create the key pair in our laptop and then let AWS know the public key, see notes "Key pair - ssh" and AI suggest how to pass the value to AWS using TF 💡

Create the key pair on your laptop:

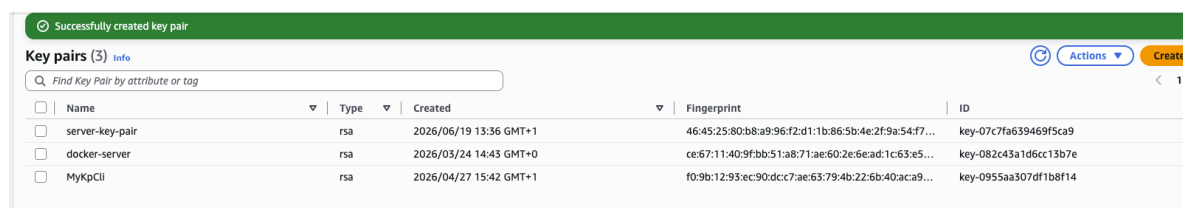
- You generate both keys locally (e.g. ssh-keygen)
- You upload the public key to AWS
- You keep the private key on your laptop
- This is actually considered more secure because your private key never travels over the internet

In Terraform you'd do it like this:

```
resource "aws_key_pair" "ssh-key" {
  key_name   = "server-key"
  public_key = file("~/ssh/id_rsa.pub")
}
```

This uploads your local public key to AWS without ever exposing your private key.

For this one I decided to follow Nana's demo, so I created the key pair with AWS and the PEM file was downloaded automatically



Now I move the file to my .ssh folder

-> mv ~/Downloads/server-key-pair.pem ~/.ssh/

```
~ ls -la
.
..
.CFUserTextEncoding
.DS_Store
.ServiceHub
.Trash
.android
.aspNet
.aws
.bash_history
.bash_profile
.cache
.cargo
.claude
.claude.json
.config
.cups
.docker
.dotnet
.emacs.d
.gitconfig
.gradle
.h2.server.properties
.homebrew
.ipynb_checkpoints
.ipynb
.jbang
.jupyter
.kube
.lessshst
.local
.m2
.matplotlib
.minikube
.npm
.npmrc
.nuget
.oh-my-zsh
.pgadmin
.rustup
.shell.pre-oh-my-zsh
.sonarlint
.ssh
.templateengine
.terraform.d
.testcontainers.properties
.viminfo
.viminfo
.vscode
.zcompdump-Daniels-MacBook-Pro-3-5-9
.zcompdump-Daniels-MacBook-Pro-3-5-9.zwc
.zsh_history
.zshrc
.zshrc.pre-oh-my-zsh
AI
Applications
Desktop
Documents
Downloads
Head-First-Java-master
Head-First-3rd-Edition
Library
Module_10_Chapter-12
Module_10_Helm
Module_10_Kubernetes
Module_10_chapter-17
Module_10_chapter-21
Module_10_chapter-24_jenkins
Module_11_Chapter_2
Module_11_Chapter_6
Module_9_AWS_CLI
Module_9_Chapter_11
Module_9_Chapter_8
Module_9_react-nodejs-example
Movies
Music
MyKpCLI.pem
Pictures
Public
VirtualBox VMs
Zotero
analysing_sorts_C
autoscaler_logs.txt
build-tools-exercises
changePwdPolicy.json
cloud-basics-exercises
cloud-basics-exercises_old
code
configmap-and-secret-volume-types
docker-exercises
eks
java-app
java-app-gradle
java-maven-app
java-react-example
js-app
logs
nginx-deployment-result.yaml
nginx-deployment.yaml
nginx-service.yaml
notebooks
package-lock.json
react-nodejs-example
software
terraform
trelo.json
~ mv ~/Downloads/server-key-pair.pem ~/.ssh/
```

And it is done

```
~ ls .ssh
config      docker-server.pem  id_ed25519      id_ed25519.pub  known_hosts  known_hosts.old  server-key-pair.pem
```

Next, we need to restrict permissions on .pem file. We want only the owner to have read permission and that's it.

-> `chmod 400 .ssh/server-key-pair.pem`

```
~ ls -l .ssh/server-key-pair.pem
-rw-r--r--@ 1 astrid staff 1674 19 Jun 13:36 .ssh/server-key-pair.pem
~ chmod 400 .ssh/server-key-pair.pem
~ ls -l .ssh/server-key-pair.pem
-r-----@ 1 astrid staff 1674 19 Jun 13:36 .ssh/server-key-pair.pem
```

This is a requirement, if we don't do it AWS will reject the SSH request

```
% chmod 400 ~/.ssh/server-key-pair.pem
nana@macbook /Users/nana/terraform [master]
% ls -l ~/.ssh/server-key-pair.pem
-r-----@ 1 nana staff 1674 13 Oct 13:15 /Users/nana/.ssh/server-key-pair.pem
nana@macbook /Users/nana/terraform [master]
%
```

 AWS rejects SSH request, if permissions not set correctly

Now we can use the key to create the EC2. To make it possible for us to use the pem file to SSH into the EC2 instance

```

118 resource "aws_instance" "myapp-server" {
119     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
120     instance_type = var.instance_type
121
122     subnet_id = aws_subnet.myapp-subnet-1.id
123     vpc_security_group_ids = [aws_default_security_group.default-sg.id]
124     availability_zone = var.avail_zone
125
126     associate_public_ip_address = true
127     key_name = "server-key-pair"
128 }
129

```

Next we add the 'tags'

```

118 resource "aws_instance" "myapp-server" {
119     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
120     instance_type = var.instance_type
121
122     subnet_id = aws_subnet.myapp-subnet-1.id
123     vpc_security_group_ids = [aws_default_security_group.default-sg.id]
124     availability_zone = var.avail_zone
125
126     associate_public_ip_address = true
127     key_name = "server-key-pair"
128
129     tags = {
130         Name = "${var.env_prefix}-server"
131     }
132 }

```

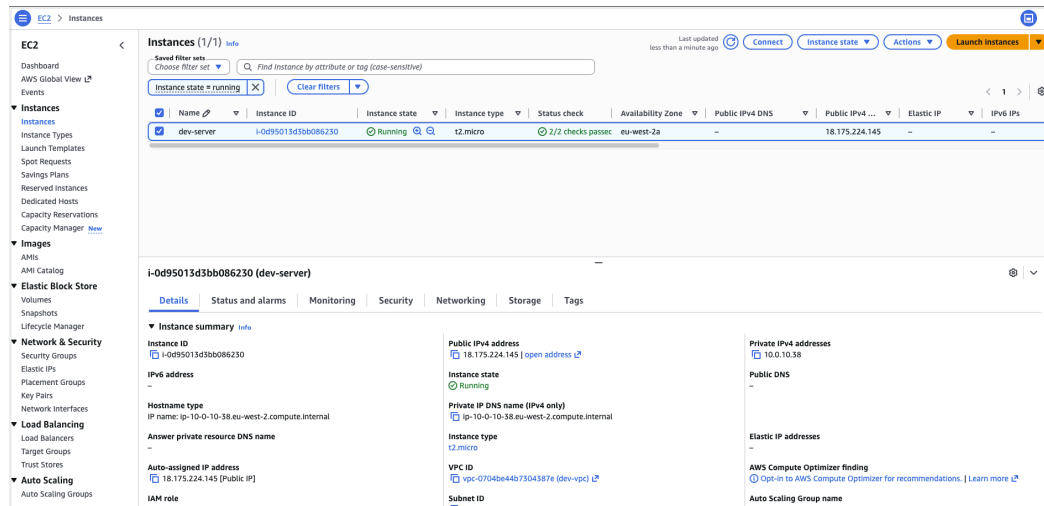
-> terraform plan

```
# aws_instance.myapp-server will be created
+ resource "aws_instance" "myapp-server" {
  + ami                                = (sensitive value)
  + arn                                = (known after apply)
  + associate_public_ip_address       = true
  + availability_zone                  = "eu-west-2a"
  + disable_api_stop                   = (known after apply)
  + disable_api_termination            = (known after apply)
  + ebs_optimized                      = (known after apply)
  + enable_primary_ipv6                = (known after apply)
  + force_destroy                      = false
  + get_password_data                  = false
  + host_id                           = (known after apply)
  + host_resource_group_arn            = (known after apply)
  + iam_instance_profile                = (known after apply)
  + id                                = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle                  = (known after apply)
  + instance_state                     = (known after apply)
  + instance_type                      = "t2.micro"
  + ipv6_address_count                  = (known after apply)
  + ipv6_addresses                     = (known after apply)
  + key_name                           = "server-key-pair"
  + monitoring                         = (known after apply)
  + outpost_arn                        = (known after apply)
  + password_data                       = (known after apply)
}
```

-> terraform apply

```
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Still creating... [00m20s elapsed]
aws_instance.myapp-server: Creation complete after 22s [id=i-0d95013d3bb086230]
```

It takes bait of time to create the EC2 instance and I check in AWS UI



Our subnet has this range "10.0.10.0/24"



And the private address of our new EC2 is 10.0.10.38 which comes from the range above.

And we also got a public IP address that we can use with the browser and to SSH into the instance.

-> `ssh -i ~/.ssh/server-key-pair.pem ec2-user@18.175.224.145`

-i -> stands for identity file and requires we pass the path to the identity file which is ~/.ssh/server-key-pair.pem

Ec2-user -> is the default user for Amazon Linux instances

18.175.224.145 -> public IP

"SSH into the server at 18.175.224.145, log in as ec2-user, and use this private key to prove my identity."

```
→ ~ ssh -i ~/.ssh/server-key-pair.pem ec2-user@18.175.224.145
The authenticity of host '18.175.224.145 (18.175.224.145)' can't be established.
ED25519 key fingerprint is SHA256:tXSo/tVYeQjzjBySGAk8cuNOv7HPLueWIS1zWENk0a0.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.175.224.145' (ED25519) to the list of known hosts.

      #_
    ~\  ####_      Amazon Linux 2023
    ~ ~ \#####\
    ~ ~  \###|
    ~ ~   \#/___  https://aws.amazon.com/linux/amazon-linux-2023
    ~ ~    V~' '->
    ~ ~ ~
    ~ ~ ._. /
    ~ ~ _/_/_/
    ~ ~ _/m/'

[ec2-user@ip-10-0-10-38 ~]$
```

And we get into the EC2 instance and we now are using the internal private IP address '10.0.10.38'

```
[ec2-user@ip-10-0-10-38 ~]$
```

Automate SSH key pair

Creating key pairs the way we just did is not efficient if we need many of them.

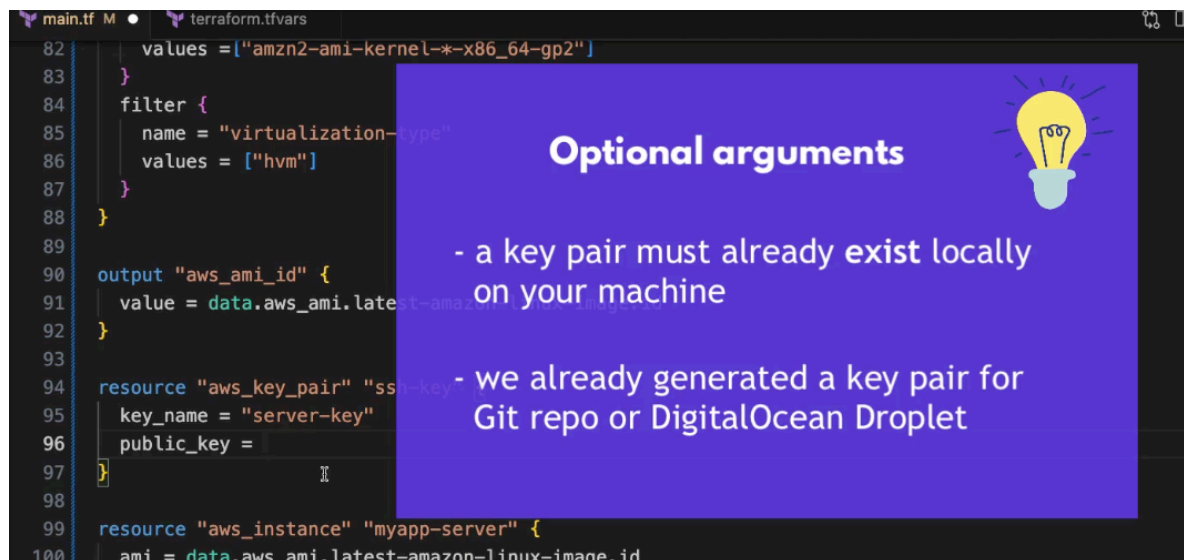
It is better to use TF instead of using the AWS console -> "aws_key_pair"

```
118 resource "aws_key_pair" "ssh-key" {
119     key_name     = "server-key"
120     public_key =
121 }
```

You, Moments ago • Uncommitted changes

I already have a public key file in my laptop as I already created key pair For Git repo and for DigitalOcean droplet.

I used -> ssh-keygen to generate the key for the first time.



Optional arguments

- a key pair must already exist locally on your machine
- we already generated a key pair for Git repo or DigitalOcean Droplet

```

82     values = ["amzn2-ami-kernel-*-x86_64-gp2"]
83   }
84   filter {
85     name = "virtualization-type"
86     values = ["hvm"]
87   }
88 }
89
90 output "aws_ami_id" {
91   value = data.aws_ami.latest-amazon-linux-image.id
92 }
93
94 resource "aws_key_pair" "ssh-key" {
95   key_name = "server-key"
96   public_key =
97 }
98
99 resource "aws_instance" "myapp-server" {
100   ami = data.aws_ami.latest-amazon-linux-image.id

```

-> ls .ssh | grep .pub

```

~ ls .ssh
config      docker-server.pem  id_ed25519        id_ed25519.pub    known_hosts      known_hosts.old   server-key-pair.pem
~ ls .ssh | grep .pub
id_ed25519.pub
~

```

Nana uses RSA public key but I am using Ed25519 algorithm (see notes Key pair - ssh)

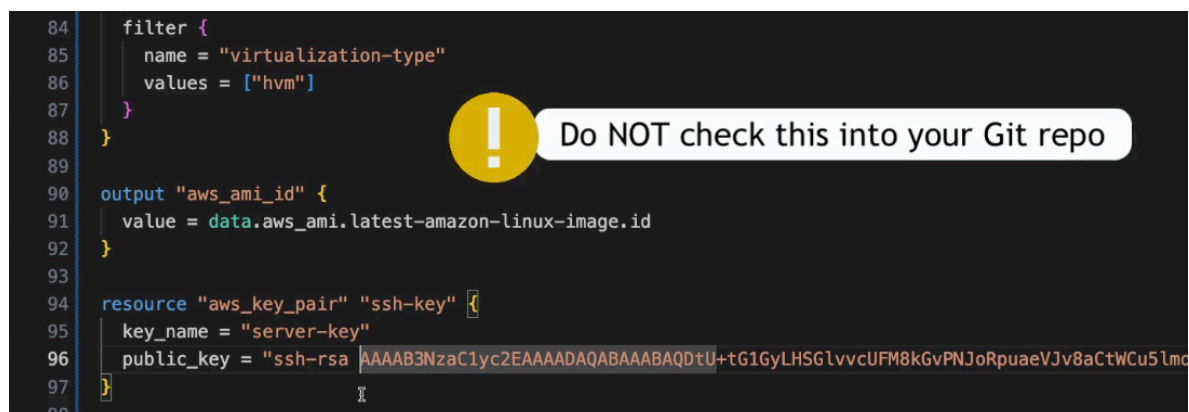
-> id_ed25519 is actually a newer and better algorithm than id_rsa.

```

nana@macbook /Users/nana/terraform [master]
% ls ~/.ssh/id_rsa
id_rsa  id_rsa.pub

```

And we can -> cat into the file and get the public key and paste it in TF file, but is bad security practice



! Do NOT check this into your Git repo

```

84   filter {
85     name = "virtualization-type"
86     values = ["hvm"]
87   }
88 }
89
90 output "aws_ami_id" {
91   value = data.aws_ami.latest-amazon-linux-image.id
92 }
93
94 resource "aws_key_pair" "ssh-key" {
95   key_name = "server-key"
96   public_key = "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDttU+tG1GyLHSGlvvcUFM8kGvPNJoRpuaeVJv8aCtWCu5lmc
97 }
98

```

So we can extract the value to TF variable file or we can reference the file location

```

92 }
93
94 resource "aws_key_pair" "ssh-key" {
95   key_name = "server-key"
96   public_key = var.my_public_key
97 }

```

Reference file location 

So we use 'file'

```

119 resource "aws_key_pair" "ssh-key" {
120   key_name = "server-key"
121   public_key = file("~/ssh/id_ed25519.pub")
122 }

```

Even better, we can be flexible with the location so we can make the location a variable

```

95 #...
96
115 data "aws_ssm_parameter" "latest-amazon-linux-image" {...}
118
119 resource "aws_key_pair" "ssh-key" {
120   key_name = "server-key"
121   public_key = file(var.public_key_location)
122 }
123
124 resource "aws_instance" "myapp-server" {
125   ami = data.aws_ssm_parameter.latest-amazon-linux-image.value

```

```

1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 avail_zone = "eu-west-2a"
4 env_prefix = "dev"
5 my_ip = "140.228.47.252/32"
6 instance_type = "t2.micro"
7 public_key_location = "/Users/astrid/.ssh/id_ed25519.pub"

```

Now I can replace the hardcoded key_nme in line 133 for the reference t"aws_key_pair.ssh-key.key_name"

```

119 resource "aws_key_pair" "ssh-key" {
120   key_name = "server-key"
121   public_key = file(var.public_key_location)
122 }
123
124 resource "aws_instance" "myapp-server" {
125   ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
126   instance_type = var.instance_type
127
128   subnet_id = aws_subnet.myapp-subnet-1.id
129   vpc_security_group_ids = [aws_default_security_group.default-sg.id]
130   availability_zone = var.avail_zone
131
132   associate_public_ip_address = true
133   key_name = aws_key_pair.ssh-key.key_name
134
135   tags = {
136     Name = "${var.env_prefix}-server"
137   }
138 }

```

-> terraform plan

We see that the server will be re-created because the change in key pair requires to destroy the current EC2 and create a new one with the new key pair.

```
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
-/+ destroy and then create replacement

Terraform will perform the following actions:

# aws_instance.myapp-server must be replaced
-/+ resource "aws_instance" "myapp-server" {
  ~ arn                  = "arn:aws:ec2:eu-west-2:706976333940:instance/i-0d95013d3bb086230" -> (known after apply)
  ~ disable_api_stop     = false -> (known after apply)
  ~ disable_api_termination = false -> (known after apply)
  ~ ebs_optimized         = false -> (known after apply)
  + enable_primary_ipv6   = (known after apply)
  - hibernation           = false -> null
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
  ~ id                    = "i-0d95013d3bb086230" -> (known after apply)
  ~ instance_initiated_shutdown_behavior = "stop" -> (known after apply)
  + instance_lifecycle    = (known after apply)
  ~ instance_state        = "running" -> (known after apply)
  ~ ipv6_address_count    = 0 -> (known after apply)
  ~ ipv6_addresses        = [] -> (known after apply)
  ~ key_name               = "server-key-pair" -> "server-key" # forces replacement
  ~ monitoring             = false -> (known after apply)
```

```
# aws_key_pair.ssh-key will be created
+ resource "aws_key_pair" "ssh-key" {
  + arn              = (known after apply)
  + fingerprint      = (known after apply)
  + id               = (known after apply)
  + key_name         = "server-key"
  + key_name_prefix  = (known after apply)
  + key_pair_id      = (known after apply)
  + key_type         = (known after apply)
  + public_key       = "ssh-ed25519 AAAAC3...adSh astridcaballero@gmail.com"
  + region           = "eu-west-2"
  + tags_all         = (known after apply)
}

Plan: 2 to add, 0 to change, 1 to destroy.
```

-> terraform apply

```

Plan: 2 to add, 0 to change, 1 to destroy.
aws_instance.myapp-server: Destroying... [id=i-0d95013d3bb086230]
aws_instance.myapp-server: Still destroying... [id=i-0d95013d3bb086230, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0d95013d3bb086230, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0d95013d3bb086230, 00m30s elapsed]
aws_instance.myapp-server: Destruction complete after 30s
aws_key_pair.ssh-key: Creating...
aws_key_pair.ssh-key: Creation complete after 0s [id=server-key]
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Still creating... [00m20s elapsed]
aws_instance.myapp-server: Still creating... [00m30s elapsed]
aws_instance.myapp-server: Creation complete after 32s [id=i-02588624ff5fb6079]

Apply complete! Resources: 2 added, 0 changed, 1 destroyed.
→ terraform git:(Module_12/Terraform_chapter-13) ×

```

I can see in AWS UI that dev-server was destroyed 'terminated' and then a new one was created 'running'

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
my-instance	i-0779fab597914e40	Stopped	t2.micro	—	eu-west-2b	—	—	—	—
my-instance	i-0c94659a8792f7a40	Stopped	t2.micro	—	eu-west-2c	—	—	—	—
dev-server	i-0d95013d3bb086230	Terminated	t2.micro	—	eu-west-2a	—	—	—	—
dev-server	i-02588624ff5fb6079	Running	t2.micro	2/2 checks passed	eu-west-2a	—	18.132.198.54	—	—

—Tangent—

I have couple of instances 'stopped' 🙌. A stopped instance means:

- The instance is not running so you're not charged for compute
- BUT you are still charged for the EBS storage (the disk attached to it)
- It's not terminated/destroyed, just paused

I have terminated them manually

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Pub
my-instance	i-0779fab597914e40	Terminated	t2.micro	—	eu-west-2b	—
	i-0c94659a8792f7a40	Terminated	t2.micro	—	eu-west-2c	—
dev-server	i-0d95013d3bb086230	Terminated	t2.micro	—	eu-west-2a	—
dev-server	i-02588624ff5fb6079	Running	t2.micro	2/2 checks passed	eu-west-2a	—

— end of tangent —

Now let's add an output to print us the public IP of the EC2 instance we created.

-> terraform state show aws_instance.myapp-server

And we get the name of the attribute -> 'public_ip'

```
placement_group          = null
placement_group_id       = null
placement_partition_number = 0
primary_network_interface_id = "eni-04b685ec7bc611ad5"
private_dns              = "ip-10-0-10-216.eu-west-2.compute.internal"
private_ip               = "10.0.10.216"
public_dns               = null
public_ip                = "18.132.198.54"
region                   = "eu-west-2"
secondary_private_ips    = []
security_groups           = []
source_dest_check         = true
```

```
114   output "ec2_public_ip" {
115       value = aws_instance.myapp-server.public_ip
116   }
```

Now how do ssh into the EC2 instance if we don't have pem file this time?

-> ssh -i ~/.ssh/id_ed25519 ec2-user@18.132.198.54

~/.ssh/id_ed25519 -> path to private key (we pass the public - pub ket to TF but we use the private to ssh)

```
> ~ ls .ssh
config          docker-server.pem  id_ed25519
> ~ ls .ssh | grep .pub
id_ed25519.pub
> ~ ssh -i ~/.ssh/id_ed25519 ec2-user@18.132.198.54
```

And we are in the new EC2 instance

```

➔ ~ ssh -i ~/.ssh/id_ed25519 ec2-user@18.132.198.54
The authenticity of host '18.132.198.54 (18.132.198.54)' can't be established.
ED25519 key fingerprint is SHA256:SNJtm+fUQMXrGvPLJ3n73KFeNjticQCbNHPA6ggPM/k.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.132.198.54' (ED25519) to the list of known hosts.

      #_
    ~\  #####_      Amazon Linux 2023
    ~ ~ \#####\
    ~ ~  \###|
    ~ ~   \#/  ___  https://aws.amazon.com/linux/amazon-linux-2023
    ~ ~    V~'  '->
    ~ ~
    ~ ~
    ~ ~. _ . _/
    ~ ~ _/ _/
    ~ ~ _/m/'

[ec2-user@ip-10-0-10-216 ~]$ █

```

We can also ssh ignoring the path -> ~/.ssh/id_ed25519 as this is a default path
-> ssh ec2-user@18.132.198.54

```

➔ ~ ssh ec2-user@18.132.198.54

      #_
    ~\  #####_      Amazon Linux 2023
    ~ ~ \#####\
    ~ ~  \###|
    ~ ~   \#/  ___  https://aws.amazon.com/linux/amazon-linux-2023
    ~ ~    V~'  '->
    ~ ~
    ~ ~
    ~ ~. _ . _/
    ~ ~ _/ _/
    ~ ~ _/m/'

Last login: Fri Jun 19 14:19:12 2026 from 140.228.47.252
[ec2-user@ip-10-0-10-216 ~]$ █

```

So we can remove the pem file from our .ssh folder, we don't need it we have our own key pair.

-> rm .ssh/server-key-pair.pem

```

➔ ~ rm .ssh/server-key-pair.pem
override r----- astrid/staff for .ssh/server-key-pair.pem? yes
➔ ~ █

```

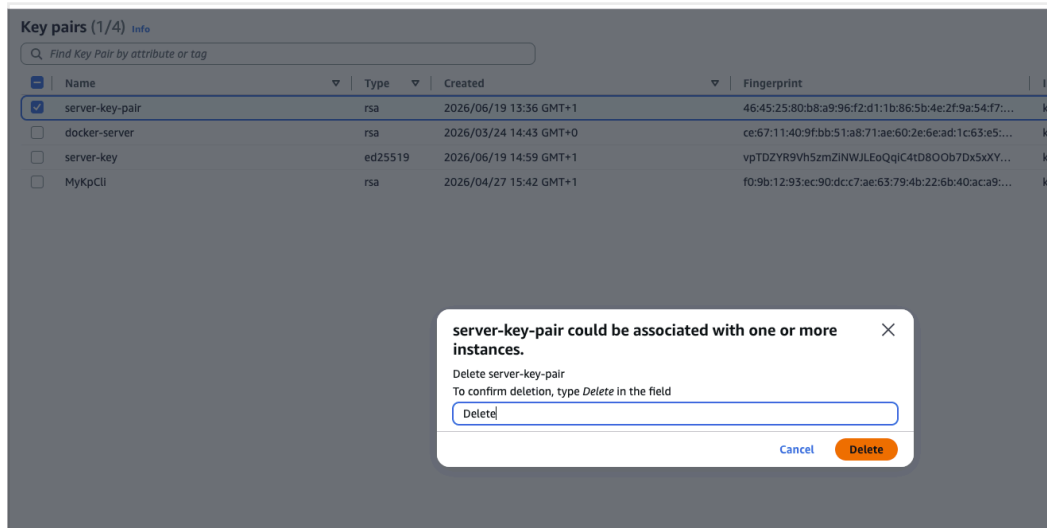
And the pem file is gone

```

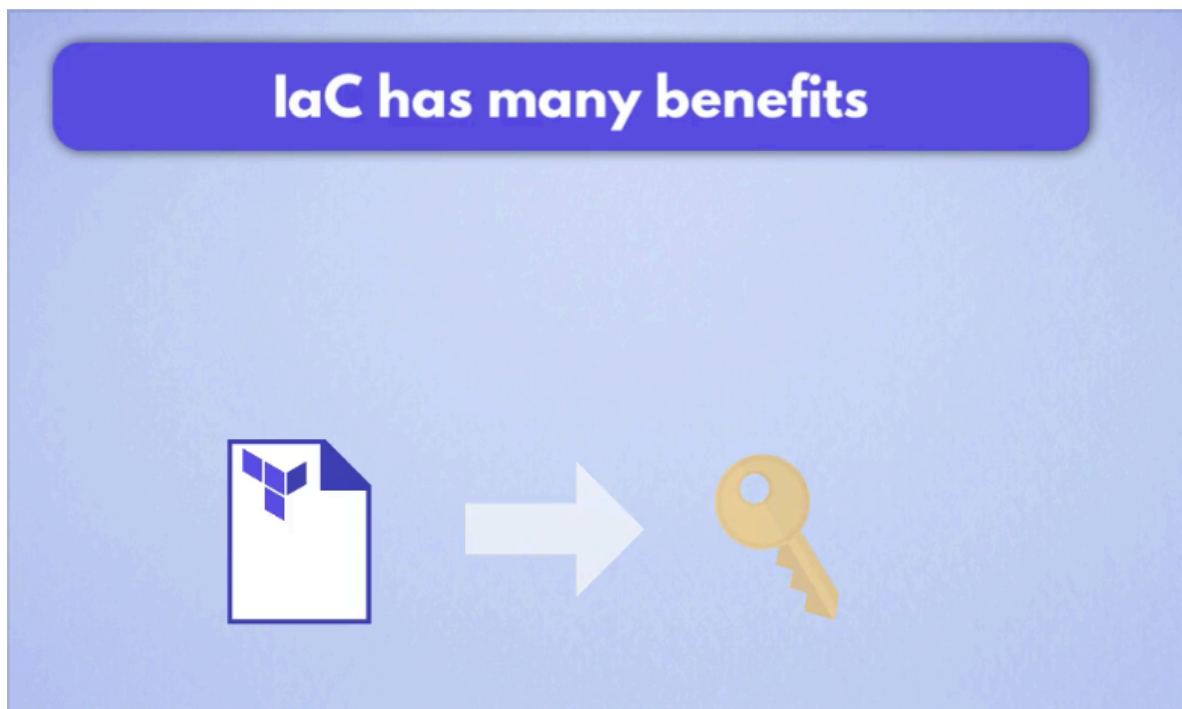
➔ ~ ls .ssh
config          docker-server.pem id_ed25519      id_ed25519.pub  known_hosts     known_hosts.old
➔ ~ █

```

We can also delete the key pair from AWS UI



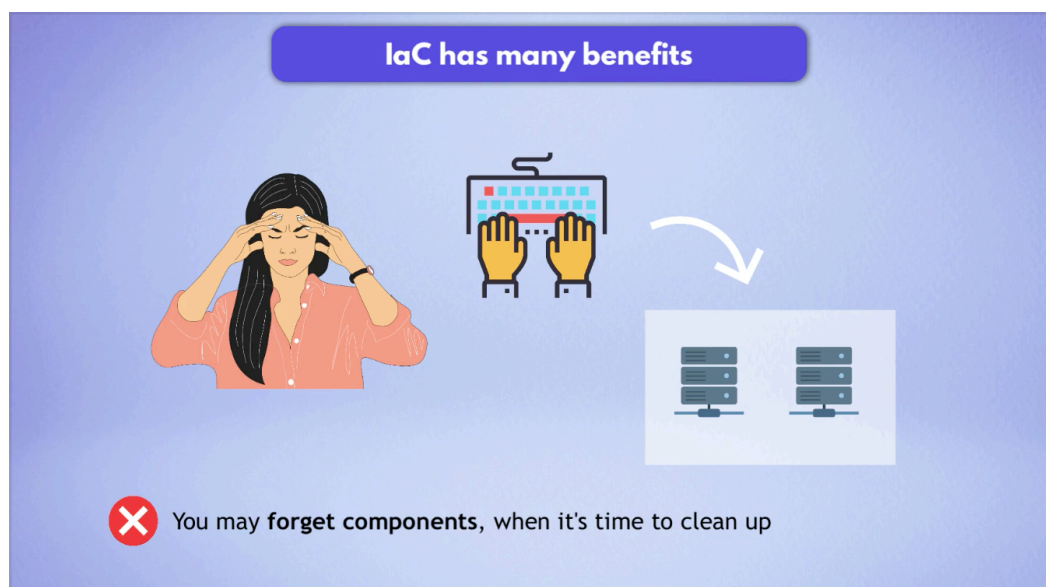
So we automated the creation of the ssh key for the EC2 instance. -> IaC has many benefits



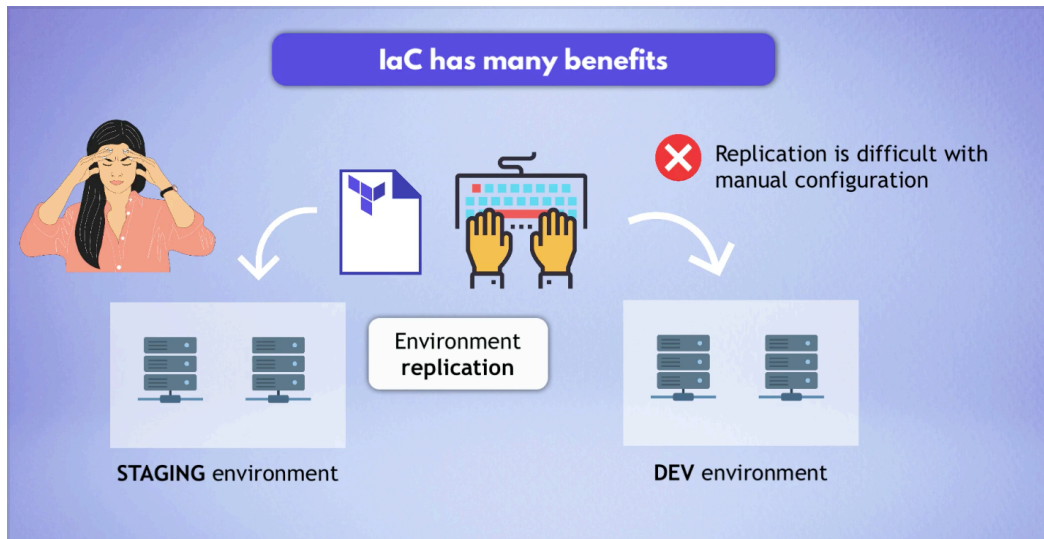
Automate as much as possible with TF and as little as possible manually or ideally nothing manually



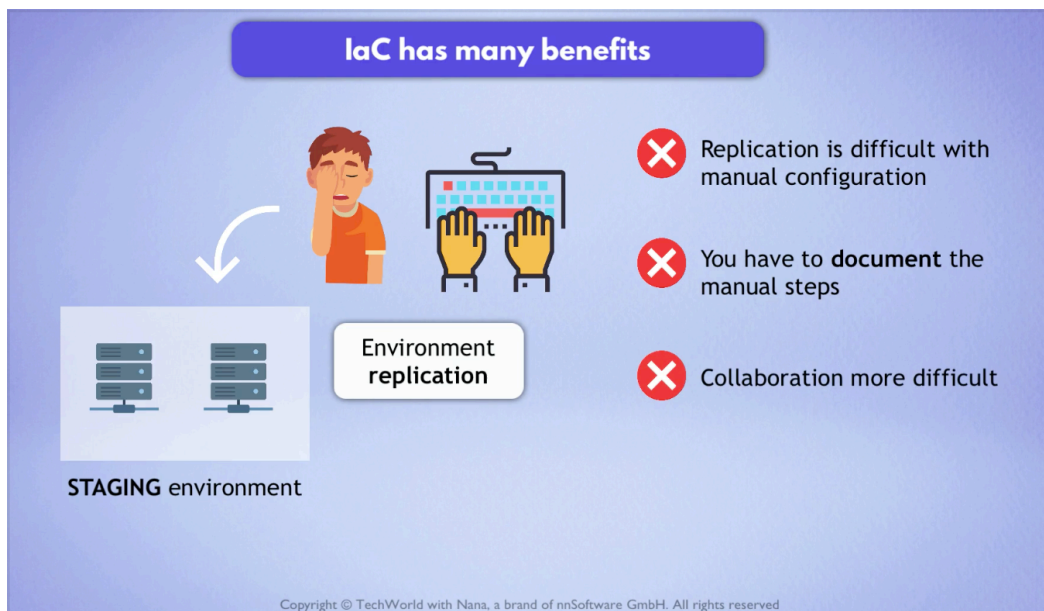
When doing things Manually -> easy to forget what we did when we need to clean up



Replicating environments is more challenging if done manually



So then because we did this manually we have to document the steps and make sure we share with the team what we did



Next

5. Deploy docker container for nginx